

Testautomatisierung in der Automobilindustrie: Vom Jenkinsstein zur wartbaren Testplattform

Karsten Günther, Alexandru Maxiniuc - Marquardt GmbH

Vorspann

Testautomatisierung über alle Teststufen hinweg ist in der Automobilindustrie unverzichtbar. Doch was passiert, wenn die gesamte Testlogik in CI-Pipeline-DSL implementiert wird? In diesem Beitrag berichten wir aus 20 Jahren Praxis in der Embedded-Softwareentwicklung, wie wir von einem monolithischen "Jenkinsstein" - tausenden Zeilen Groovy-Code, der Build-, Test- und Deployment-Logik vermischte - zu einer modularen Testplattform auf Basis von Python und Pytest gefunden haben. Der Schlüssel: Quality Gates als einfache Pytest-Marker-Selektion, die lokal und in CI identisch funktionieren.

Das Problem: Testlogik am falschen Ort

Unsere CI-Reise begann 2005 in der Automobilzulieferindustrie. Der Ausgangspunkt war ernüchternd: Kein einziger Unit-Test im Repository. Getestet wurde explorativ am Target - im Labor oder direkt im Fahrzeug, mit Messtechnik, Debugger und Oszilloskop. Keine Testautomatisierung, nur Nightly Builds mit dem höchsten Qualitätskriterium "Software linkbar". Wir nannten diesen Zustand "Continuous Kind im Brunnen" - reaktiv statt präventiv.

Die ersten Verbesserungsversuche brachten Unit-Tests auf Basis eines eigenen CUnit-Frameworks und Jenkins als CI-Server. Doch mit wachsenden Anforderungen - SIL-Tests (Software-in-the-Loop), HiL-Tests (Hardware-in-the-Loop), statische Codeanalyse - entstand eine fatale Entwicklung: Für jede Testart wurde ein eigener Jenkins-Freestyle-Job erstellt, später wurden diese in immer komplexere Groovy-Pipelines konsolidiert.

Das Ergebnis war unser "Jenkinsstein": Ein Monster aus zehntausenden Zeilen Jenkins-Pipeline-DSL, das als Buildsystem, Testsystem, Deploymentsystem und Monitoringsystem gleichzeitig fungierte. Die Pipeline enthielt die gesamte Geschäftslogik - welche Varianten gebaut werden, welche Tests für welche Teststufe laufen, wie Ergebnisse aggregiert werden. Shared Libraries und Plugins machten das Chaos komplett.

Die Konsequenzen waren gravierend:

- **Nicht reproduzierbare Testergebnisse:** Fehler, die in CI auftraten, konnten lokal nicht nachgestellt werden, weil die Testlogik nur in der Pipeline existierte.

- **Unwartbarer Code:** Groovy-DSL-Code in Jenkins ist schwer testbar, schwer zu debuggen und für die meisten Entwickler unverständlich.
- **Hoher Wartungsaufwand:** Zwei Scrum-Teams waren zu mindestens 50% mit der Pflege der Pipelines beschäftigt.
- **Kein Shift Left:** Entwickler konnten Quality Gates nicht lokal ausführen und erhielten Feedback erst nach dem Push.

Die Erkenntnis: CI und lokal unterscheiden sich nur in der Orchestrierung

Der Wendepunkt kam mit einer simplen Erkenntnis: CI-Umgebungen und lokale Entwicklermaschinen unterscheiden sich primär in der *Orchestrierung*, nicht in der eigentlichen Test- und Build-Ausführung. Ein Build, ein Unit-Test, eine statische Analyse - all das sollte mit denselben Kommandos funktionieren, egal ob lokal oder auf einem CI-Agent.

Daraus leiteten wir fünf Architekturprinzipien ab:

1. **Separation of Concerns:** Die Pipeline orchestriert nur - keine Geschäftslogik in Pipeline-DSL.
2. **Local-First Development:** Jenkins führt exakt dieselben Kommandos aus, die Entwickler lokal nutzen.
3. **Bootstrapping:** Build-Skripte lösen alle Abhängigkeiten selbst auf.
4. **Unified Build System:** CMake als Meta-Buildsystem für alle Varianten und Artefakte.
5. **Quality Gates als Testselektion:** Verschiedene Teststufen sind lediglich unterschiedliche Pytest-Marker-Selektionen.

Die Lösung: Unsere SPLE-Plattform

Mit diesen Prinzipien im Kopf entwarfen wir eine modulare Internal Developer Platform (IDP) für Software Product Line Engineering (SPLE). Unsere SPLE-Plattform verfolgt vier **Hauptziele**:

- **Shift Left:** Tests so früh wie möglich im Entwicklungsprozess - nicht erst im Nightly Build, sondern bereits beim Pull Request.
- **Continuous Integration:** Schnelles, zuverlässiges Feedback auf Pull Requests und den Develop-Branch.
- **Reuse:** Unterstützung mehrerer Kundenprojekte mit gemeinsamen Komponenten und Variantenmanagement - eine Plattform für die gesamte Software-Produktlinie.
- **Automation:** Automatisierte Erzeugung aller benötigten Build-Artefakte und Reports.

Das Herzstück der Plattform ist die Nutzung von Pytest als universelles Test-Framework für *alle* Quality Gates - vom Build über Unit-Tests bis hin zu Integra-

tionstests. Jede Teststufe wird durch einen Pytest-Marker repräsentiert:

```
class Test_MyVariant:
    variant = "MyVariant"

    @pytest.mark.build
    def test_build(self):
        spl_build = SplBuild(variant=self.variant,
                              build_kit="prod", target="build")
        result = spl_build.execute()
        assert result == 0, "Build fehlgeschlagen"

    @pytest.mark.unittests
    def test_unittests(self):
        spl_build = SplBuild(variant=self.variant,
                              build_kit="test", target="unittests")
        result = spl_build.execute()
        assert result == 0, "Unit-Tests fehlgeschlagen"
```

Die Quality-Gate-Auswahl erfolgt dann durch den Trigger-Typ:

- **Pull Request:** `pytest -m "build or unittests"` - schnelle Tests für schnelles Feedback
- **Develop-Branch:** `pytest -m "build or unittests or integration"` - vollständige Testsuite
- **Nightly Build:** `pytest -m "build or unittests or integration or longrunning"` - inklusive langläufiger Tests

Damit werden Quality Gates von undurchsichtiger Pipeline-Magie zu transparenten, reproduzierbaren Testselektionen. Ein Entwickler, der einen Fehler im CI nachstellen möchte, führt lokal exakt denselben Pytest-Aufruf aus - kein Pipeline-Debugging, kein "works on my machine".

Abbildung 1 zeigt unseren Ansatz als Flussdiagramm: Die Pipeline wählt basierend auf dem Trigger-Typ (Pull Request, Main-Branch-Push, Nightly Build) lediglich ein Quality Gate als Menge von Pytest-Markern aus und orchestriert die parallele Testausführung über mehrere Agents.

flowchart TD

```
subgraph "Quality Gate Selection"
    C1["What to test?"] --> C2["Trigger Type"]
    C2 -->|PR| C3["⚡ Quick Tests"]
    C2 -->|Main Branch| C4["Full Tests"]
    C2 -->|Nightly| C5["Long Tests"]
end

C3 --> C6["Start Parallel Execution"]
C4 --> C6
```

```

C5 --> C6

subgraph "□ Test Execution"
  subgraph "Agent 1"
    M1A["□ Checkout Code"] --> M1B["□ Installation of Dependencies"]
    M1B --> M1C["□ Execute Tests"]
    M1C --> M1D["□ Deploy Test Results"]
  end

  M2[". . ."]

  subgraph "Agent n"
    M3A["□ Checkout Code"] --> M3B["□ Installation of Dependencies"]
    M3B --> M3C["□ Execute Tests"]
    M3C --> M3D["□ Deploy Test Results"]
  end
end

C6 --> M1A
C6 --> M3A

M1D --> C7["□ Wait & Collect<br/>Overall Status"]
M3D --> C7

%% Style to make an element transparent
classDef transparent fill:transparent,stroke:transparent
class M2 transparent

```

Abbildung 1: SPLE-Pipeline mit Quality-Gate-Selektion und paralleler Testausführung

Der Technologie-Stack

Die SPLE-Plattform setzt auf einen bewusst schlanken Stack:

- **Scoop:** Windows-Paketmanager für die automatische Installation aller Toolchains - kein manuelles Setup, kein "bei mir fehlt Tool X".
- **CMake + Ninja:** CMake als Meta-Buildsystem generiert performante Ninja-Build-Dateien für alle Varianten. Ein einheitliches Buildsystem statt fragmentierter Makefiles.
- **Python + Pytest:** Alle Quality Gates sind Pytest-Tests mit Markern. Die Testlogik ist wartbarer Python-Code statt Groovy-DSL.
- **Pypline:** Unser CI-agnostischer Pipeline-Runner. Pipeline-Schritte werden als Python-Klassen implementiert und in einer YAML-Datei konfiguriert - dieselbe Pipeline läuft auf dem Entwickler-Laptop, in Jenkins und in GitHub Actions.

- **Jenkins:** Nur noch dünne Orchestrierungsschicht. Der Jenkinsfile ist minimal - er ruft Pytest mit den passenden Markern auf, mehr nicht.

Das Entscheidende ist die konsequente Trennung: Die Pipeline weiß *wann* und *wo* Tests laufen (Orchestrierung), aber nicht *was* und *wie* getestet wird (Geschäftslogik).

Ehrliche Learnings: Was wir dabei gelernt haben

Das “Law of the Instrument” ist real. Wer einen Hammer hat, für den sieht alles wie ein Nagel aus. Jenkins ist ein hervorragendes Orchestrierungswerkzeug, aber kein Buildsystem und kein Test-Framework. Wir haben Jahre gebraucht, um diese Grenze zu erkennen und konsequent einzuhalten.

Separate Repositories für Pipeline und Produktcode sind ein Anti-Pattern. Wenn die CI-Konfiguration in einem eigenen Repository liegt, entsteht eine künstliche Trennung, die Reproduzierbarkeit unmöglich macht. Pipeline-als-Code gehört ins Produktrepository.

Testautomatisierung muss lokal funktionieren. Der wichtigste Qualitätsindikator einer Testautomatisierung ist die Frage: “Kann ein Entwickler diesen Test auf seiner Maschine ausführen?” Wenn die Antwort “Nein” lautet, stimmt die Architektur nicht.

Die Plattform als Produkt behandeln. Der größte organisatorische Hebel war, unsere Testplattform als eigenes Produkt innerhalb eines Agile Release Train (SAFe) zu entwickeln. Regelmäßige Sprint Reviews mit echtem Nutzerfeedback, dediziertes Budget von der Geschäftsleitung und ein klarer Produktname schaffen Ownership und Nachhaltigkeit.

Fazit und Ausblick

Die Transformation vom Jenkinstein zur modularen Testplattform hat sich für alle Beteiligten gelohnt. Entwickler erhalten schnelles, lokal reproduzierbares Feedback. Plattform-Ingenieure pflegen wartbaren Python-Code statt tausender Zeilen Groovy-DSL. Das Management hat transparente Qualitätskriterien und einen stets releasefähigen Softwarestand.

Der Schlüssel lag nicht in einem neuen Tool, sondern in einer architektonischen Entscheidung: Testlogik gehört nicht in die Pipeline. Pytest-Marker als Abstraktion für Quality Gates sind einfach, transparent und universell - vom Unit-Test bis zum Systemtest, von der Entwicklermaschine bis zum CI-Server.

Referenzen

[1] Pytest: <https://docs.pytest.org/en/>

[2] CMake: <https://cmake.org/>

[3] Jenkins Pipeline Best Practices: <https://www.jenkins.io/doc/book/pipeline/pipeline-best-practices/>

[4] Internal Developer Platform: <https://internaldeveloperplatform.org/>

[5] Scoop Windows Package Manager: <https://scoop.sh/>

[6] Scaled Agile Framework (SAFe): <https://www.scaledagileframework.com/>

Kurzbiografie

Karsten Günther ist seit 25 Jahren leidenschaftlicher Softwareingenieur und kann auf Erfahrungen im Bereich der Signalverarbeitung und eingebetteten Echtzeitsysteme, in der Automatisierung von Entwicklungsprozessen, sowie in der Entwicklung von Methoden, Werkzeugen und Pipelines für Continuous Integration (CI) zurückgreifen. Er war und ist als Ingenieur, Tester und Architekt in der Automobilbranche unterwegs und arbeitet zurzeit als Product Owner der SPLE-Plattform im Rhein-Main Team (RMT) der Marquardt GmbH, wo er sich intensiv mit internen Entwicklungsplattformen und Software Product Line Engineering (SPLE) beschäftigt.

Alexandru Maxiniuc bringt fast 20 Jahre Erfahrung in der Automobilindustrie mit, spezialisiert auf sicherheitskritische eingebettete Software für Bremssysteme. Er begeistert sich für eingebettete Software, Build-Systeme und Software Product Line Engineering. Alexandru hat nachweislich innovative Lösungen entwickelt, kleine Softwareteams geleitet und die Einführung effizienter Entwicklungspraktiken vorangetrieben. Er entwickelt aktiv Open-Source-Tools und teilt sein Wissen gerne, um Teams bei der Verbesserung ihrer Software-Engineering-Fähigkeiten zu unterstützen - in seiner aktuellen Position als Plattformarchitekt der SPLE-Plattform im Rhein-Main Team (RMT) der Marquardt GmbH.